EWB COURSES

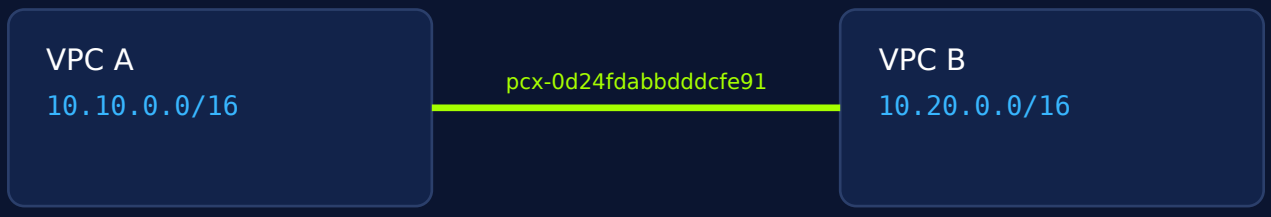
EWB CLOUD LABS
STUDENT PROJECT REPORT

BUILD / TEST / EXPLAIN

AWS

VPC Peering

Connect two VPCs.
Prove private connectivity.



2 VPCs / 3 EC2 INSTANCES / 1 PRIVATE CONNECTION

AWS ARCHITECT ASSOCIATE

Name: Lokeswari V
Batch: Batch 1
Region: Asia Pacific (Mumbai) / ap-south-1
Completed: 30 October 2026

01 / PROJECT SUMMARY

Project overview

Build a working private network connection and explain why each route and security rule is needed.

This project connects two separate Amazon VPCs with a VPC peering connection so that two private EC2 servers can talk to each other using only their private IPv4 addresses. A single public bastion host in VPC A is the only entry point for administration; both application servers have no public IP address and are reached with SSH ProxyJump through the bastion.

Item	Value
Account / Region	One lab account, one Region: Asia Pacific (Mumbai) ap-south-1, Availability Zone ap-south-1a
VPC A (ewb-vpc-a)	10.10.0.0/16 with a public subnet (10.10.1.0/24) and a private subnet (10.10.2.0/24)
VPC B (ewb-vpc-b)	10.20.0.0/16 with one private subnet (10.20.1.0/24); no internet gateway
Instances	3 x t3.micro, Amazon Linux 2023 (same AMI), same key pair (ewb-lab), login user ec2-user
Connection	Same-account, same-Region VPC peering: ewb-a-b-peer (pcx-0d24fdabbdddcfe91), status Active
Why the CIDRs do not overlap	10.10.0.0/16 and 10.20.0.0/16 are different ranges, so AWS can create the peering connection and route tables can send traffic to the correct VPC.

02 / ARCHITECTURE

Architecture diagram

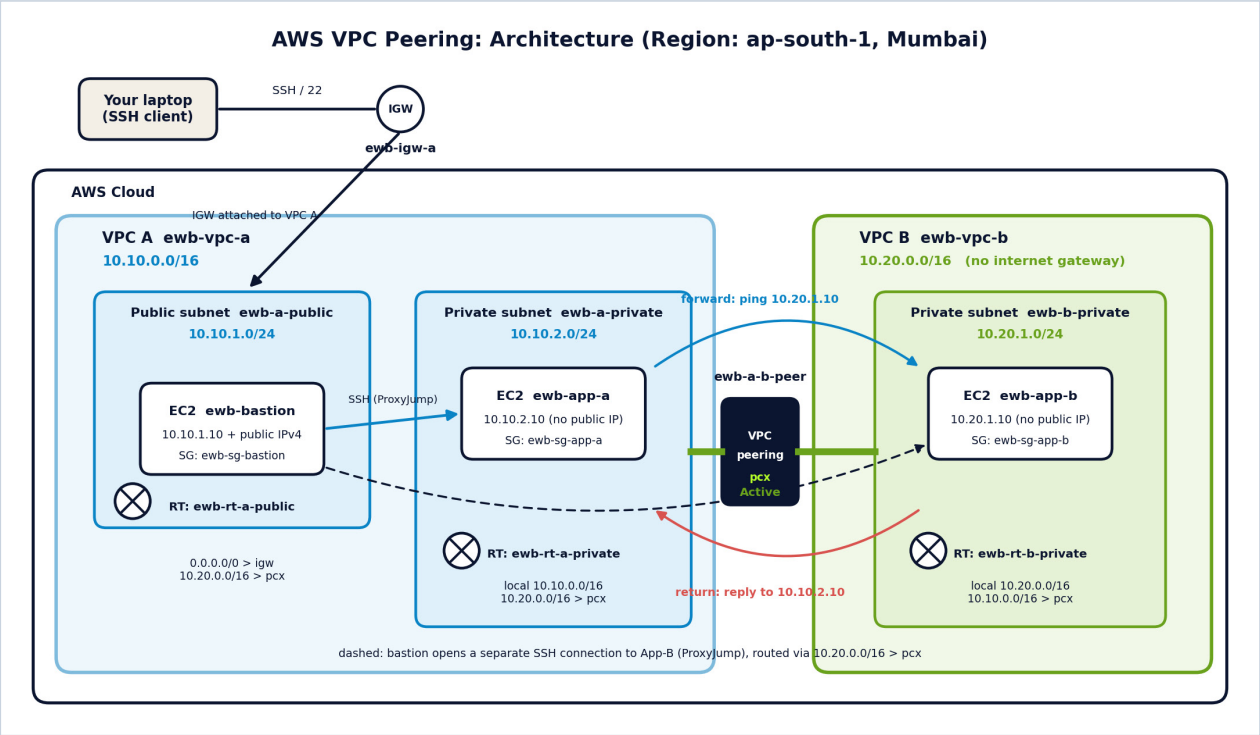


Figure 1. Architecture of the VPC peering project

Host / test / result: Diagram / design / complete

What it shows: One internet entry point (internet gateway on VPC A), a public bastion, two private application servers and one peering connection. Route tables and security groups are shown at each subnet.
Why it is needed: The diagram is the design that every later screenshot is checked against: addresses, subnets, routes and the SSH path (laptop to bastion to private hosts).

Administration path: laptop → bastion → App-A or App-B using SSH ProxyJump. **Project traffic:** App-A ↔ App-B over private IPs and the peering connection. Peering does not give VPC B internet access through VPC A (peering is not transitive for gateways).

03 / INVENTORY

Resource inventory with actual IDs

All values below were taken from my own AWS console screenshots (account number masked).

Resource (Name tag)	CIDR / private IP	Actual ID
ewb-vpc-a	10.10.0.0/16	vpc-0167a2323ec7f602f
ewb-a-public (subnet)	10.10.1.0/24	subnet-078b288a888f4a590
ewb-a-private (subnet)	10.10.2.0/24	subnet-016a19ca45ba6c444
ewb-vpc-b	10.20.0.0/16	vpc-0322851fee12de5a4
ewb-b-private (subnet)	10.20.1.0/24	subnet-0428a86f733844f2d
ewb-igw-a	attached to ewb-vpc-a	igw-078c244219093f26d
ewb-rt-a-public	10.10.0.0/16 local; 0.0.0.0/0 igw; 10.20.0.0/16 pcx	rtb-0a126363e86b584b1
ewb-rt-a-private	10.10.0.0/16 local; 10.20.0.0/16 pcx	rtb-07257232a0b38fdeb
ewb-rt-b-private	10.20.0.0/16 local; 10.10.0.0/16 pcx	rtb-04c369eee658bb812
ewb-a-b-peer	VPC A ↔ VPC B (Active)	pcx-0d24fdabbdddcfe91
ewb-sg-bastion (VPC A)	SSH from my public IP /32	sg-034f35842ec0ec854
ewb-sg-app-a (VPC A)	SSH from 10.10.1.10/32; ICMP echo from 10.20.1.10/32	sg-ID not captured yet
ewb-sg-app-b (VPC B)	SSH from 10.10.1.10/32; ICMP echo from 10.10.2.10/32	sg-0a277f3a9dc2cb94b
ewb-bastion	10.10.1.10 + public IPv4 (masked)	i-0ded36efc198c8e1a
ewb-app-a	10.10.2.10 / no public IP	i-06c909174aed02f8c
ewb-app-b	10.20.1.10 / no public IP	i-0eb16c46963e015bc
AMI (all three)	Amazon Linux 2023	ami-03765515c80e0c765

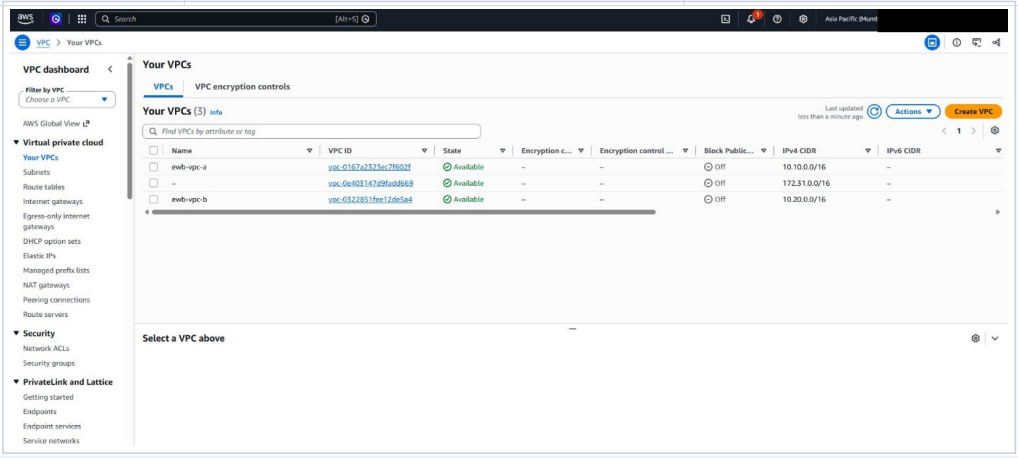


Figure 2. VPC list: ewb-vpc-a and ewb-vpc-b

Host / test / result: AWS console, VPC > Your VPCs / planning check / PASS

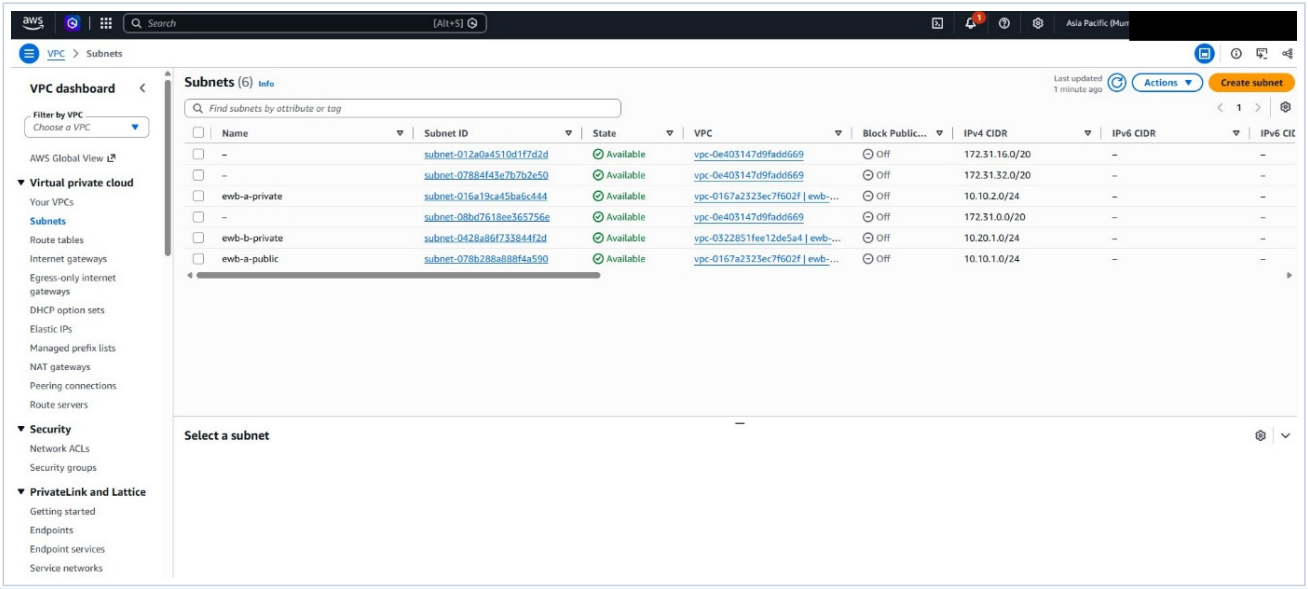


Figure 3. Subnet list with the three lab subnets

Host / test / result: AWS console, VPC > Subnets / address plan / PASS

What it shows: ewb-a-public 10.10.1.0/24, ewb-a-private 10.10.2.0/24 and ewb-b-private 10.20.1.0/24, each in its correct VPC. The 172.31.x.x subnets belong to the default VPC.

Why it is needed: Every /24 subnet must sit inside its own VPC range. Keeping public and private subnets separate lets only the bastion subnet have an internet route.

04 / ROUTING

Subnets, internet gateway and route tables

Checkpoint 01: route tables and subnet associations before peering.

Each subnet is **explicitly** associated with its own route table. The public route table has the default route 0.0.0.0/0 to the internet gateway; the private route table has only its local VPC route, so App-A cannot reach the internet or VPC B yet.

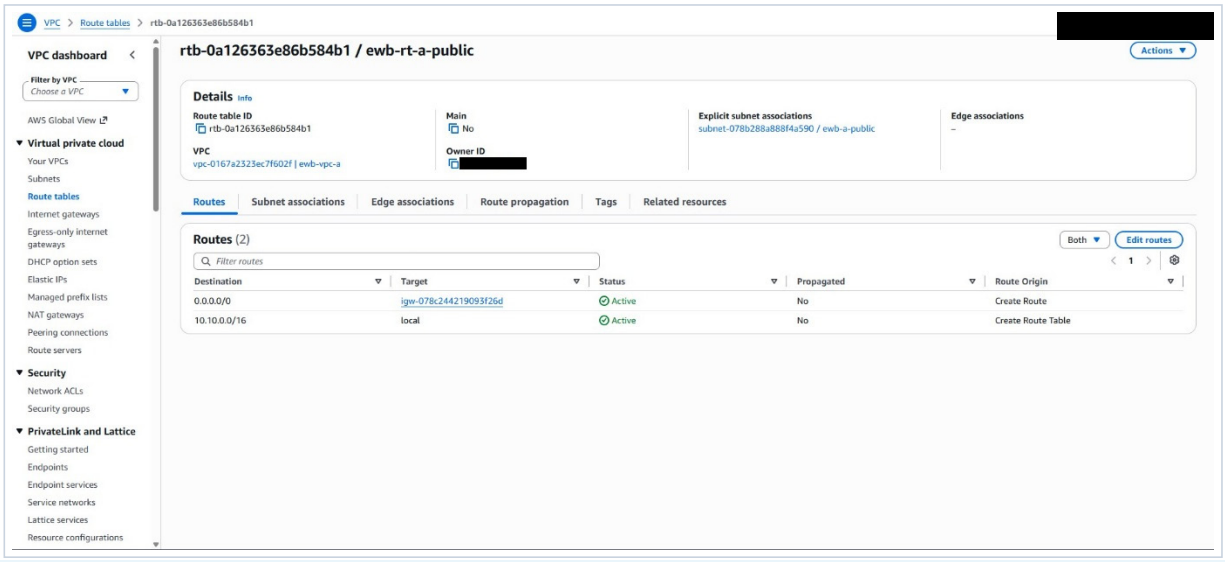


Figure 4. Public route table routes (before peering)

Host / test / result: ewb-rt-a-public / Routes tab / PASS

What it shows: Two routes: 10.10.0.0/16 → local and 0.0.0.0/0 → igw-078c244219093f26d.

Why it is needed: The internet-gateway route is what makes ewb-a-public a public subnet, so the bastion can be reached by SSH from my laptop.

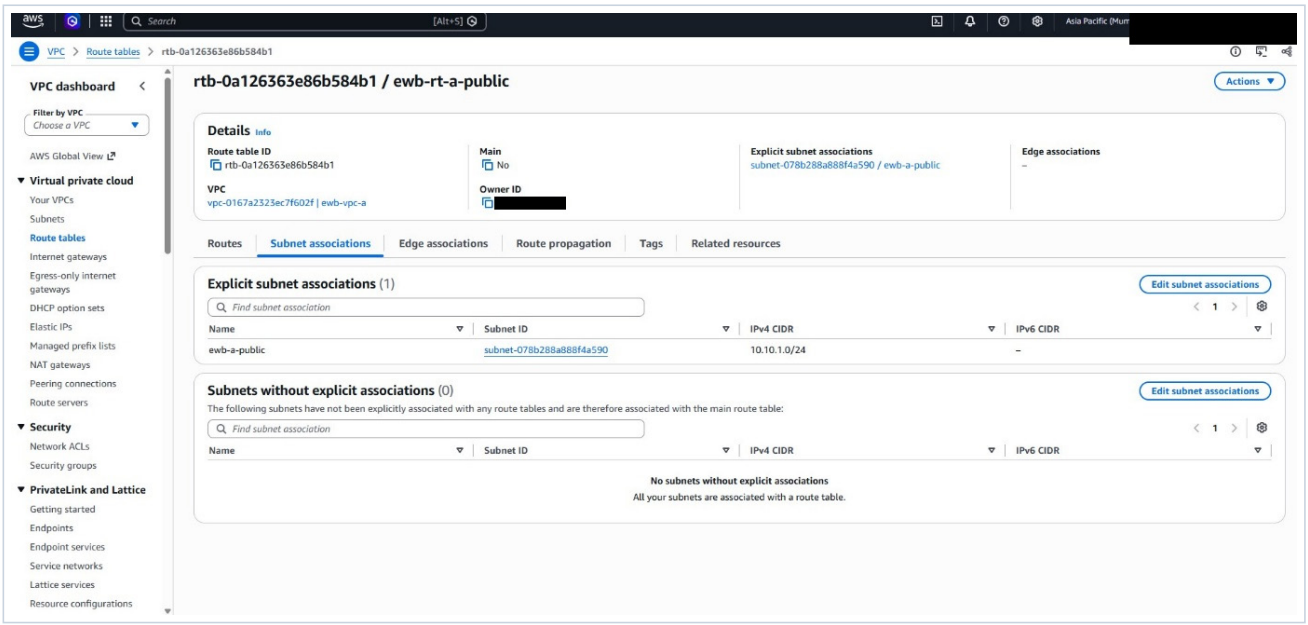


Figure 5. Public route table subnet association

Host / test / result: ewb-rt-a-public / Subnet associations tab / PASS

What it shows: Explicit association with ewb-a-public (subnet-078b288a888f4a590, 10.10.1.0/24). No subnet is left on the main route table.

Why it is needed: A route table only affects the subnets it is associated with; the explicit association is what applies the IGW route to the bastion subnet.

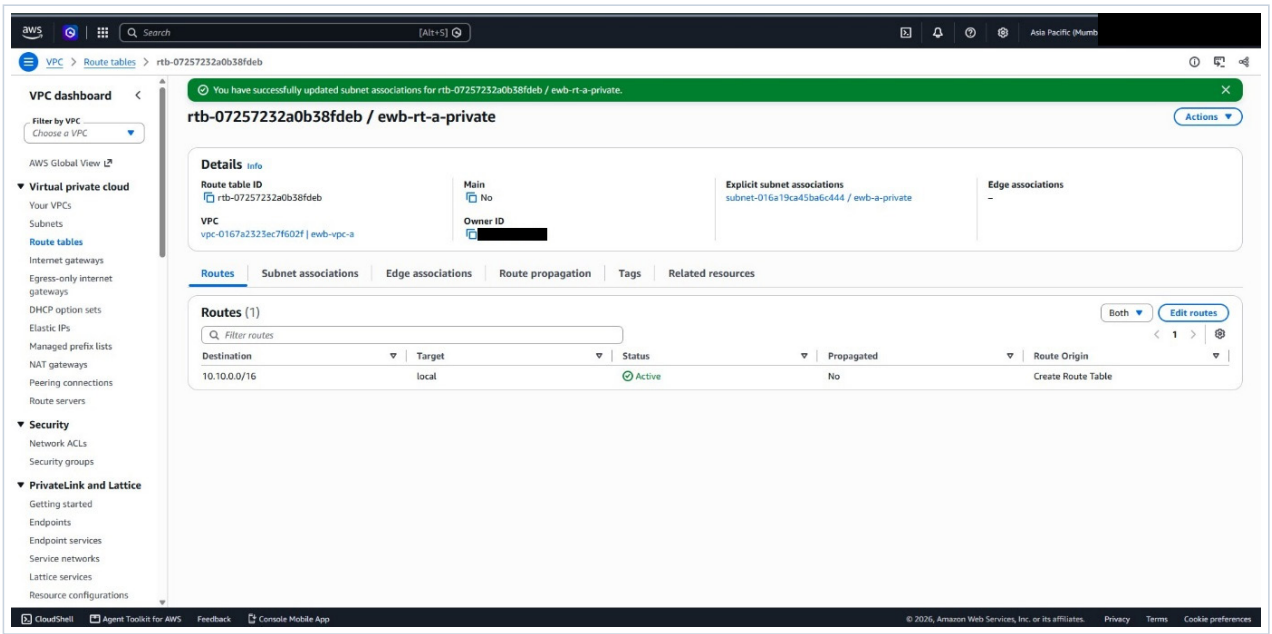


Figure 6. Private route table routes (before peering)

Host / test / result: ewb-rt-a-private / Routes tab / PASS

What it shows: Only the local route 10.10.0.0/16 → local. There is no internet route and no route to 10.20.0.0/16.

Why it is needed: This is the reason the baseline ping to App-B fails: App-A has no path to VPC B until the peering route is added.

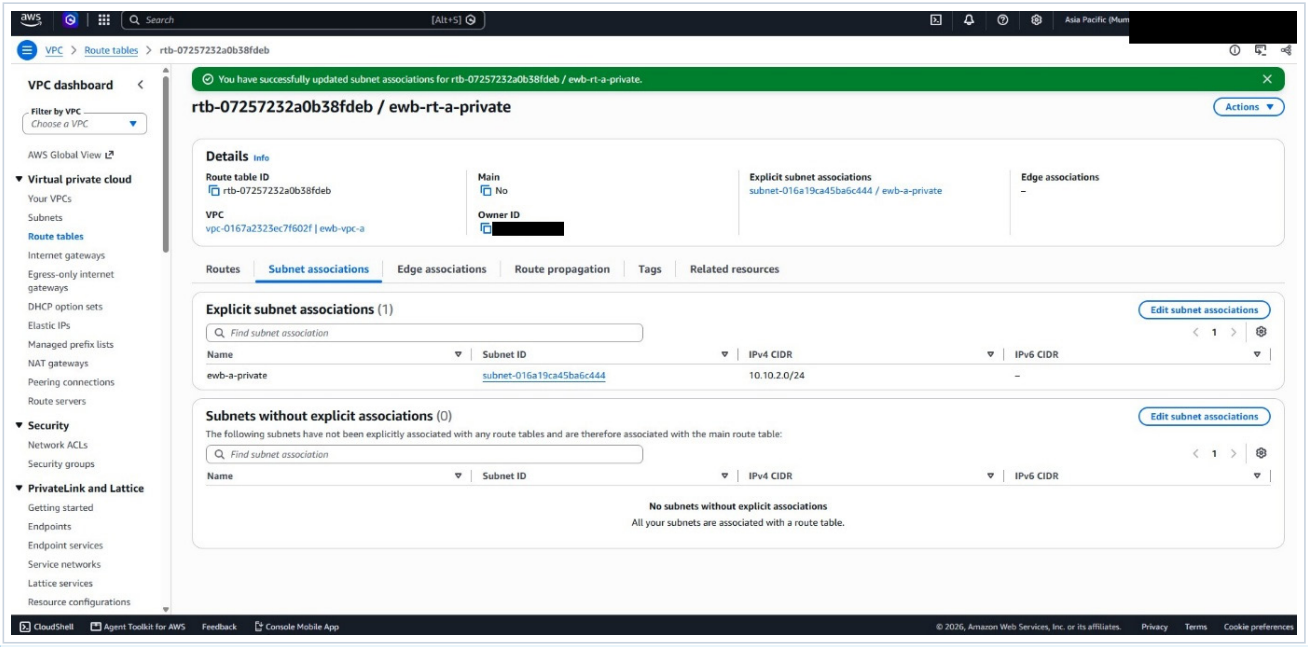


Figure 7. Private route table subnet association

Host / test / result: ewb-rt-a-private / Subnet associations tab / PASS

What it shows: Explicit association with ewb-a-private (subnet-016a19ca45ba6c444, 10.10.2.0/24).

Why it is needed: Keeps the private subnet on a route table with no internet gateway route, so App-A stays private.

05 / COMPUTE

EC2 instances

Checkpoint 02: three instances running with the correct IPs and no public IP on the application servers.

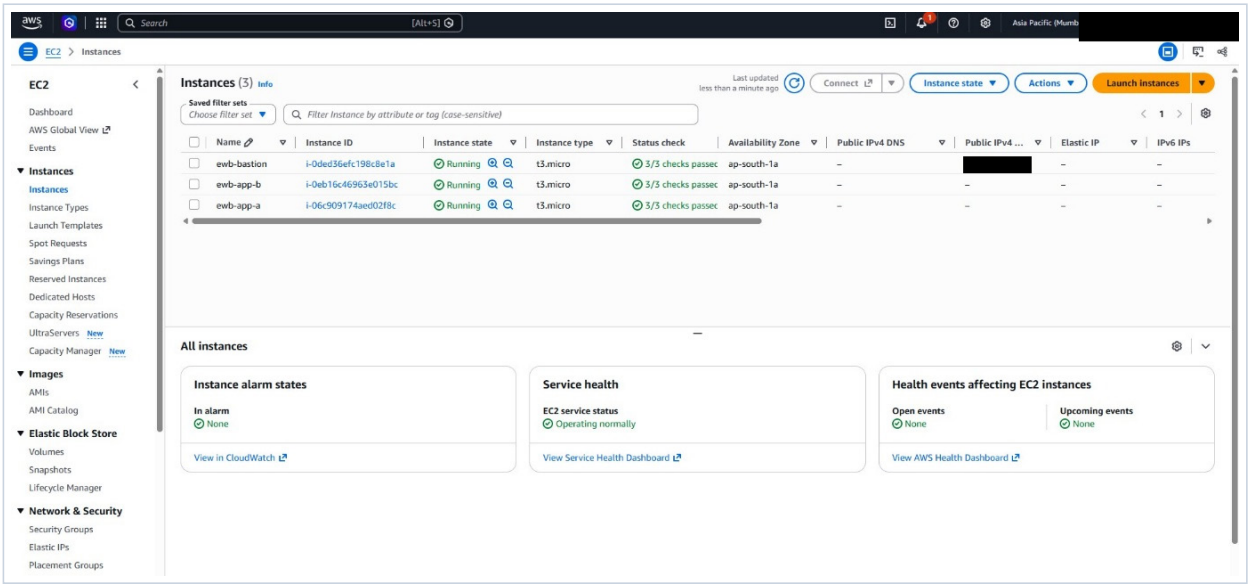


Figure 8. Instance list: all three instances running

Host / test / result: EC2 > Instances / Checkpoint 02 / PASS

What it shows: ewb-bastion, ewb-app-a and ewb-app-b are Running, type t3.micro, in ap-south-1a, with 3/3 status checks passed. Only the bastion has a Public IPv4 address (masked).

Why it is needed: Instances must pass status checks before SSH tests are valid. The empty public-IP column for the two apps shows they are not reachable from the internet.

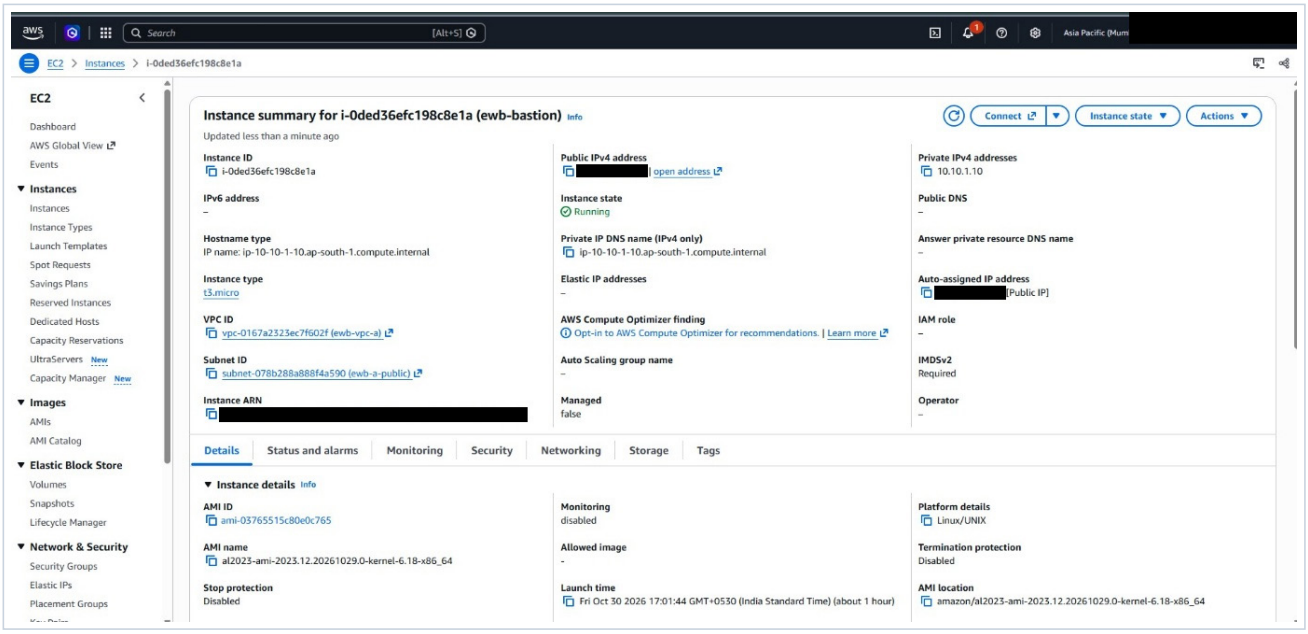


Figure 9. Bastion instance details

Host / test / result: ewb-bastion / instance details / PASS

What it shows: Private IPv4 10.10.1.10, auto-assigned public IPv4 (masked), VPC ewb-vpc-a, subnet ewb-a-public, AMI Amazon Linux 2023.

Why it is needed: The bastion is the only host with a public address. Its fixed private IP 10.10.1.10 is the source used in the SSH rules of both private security groups.

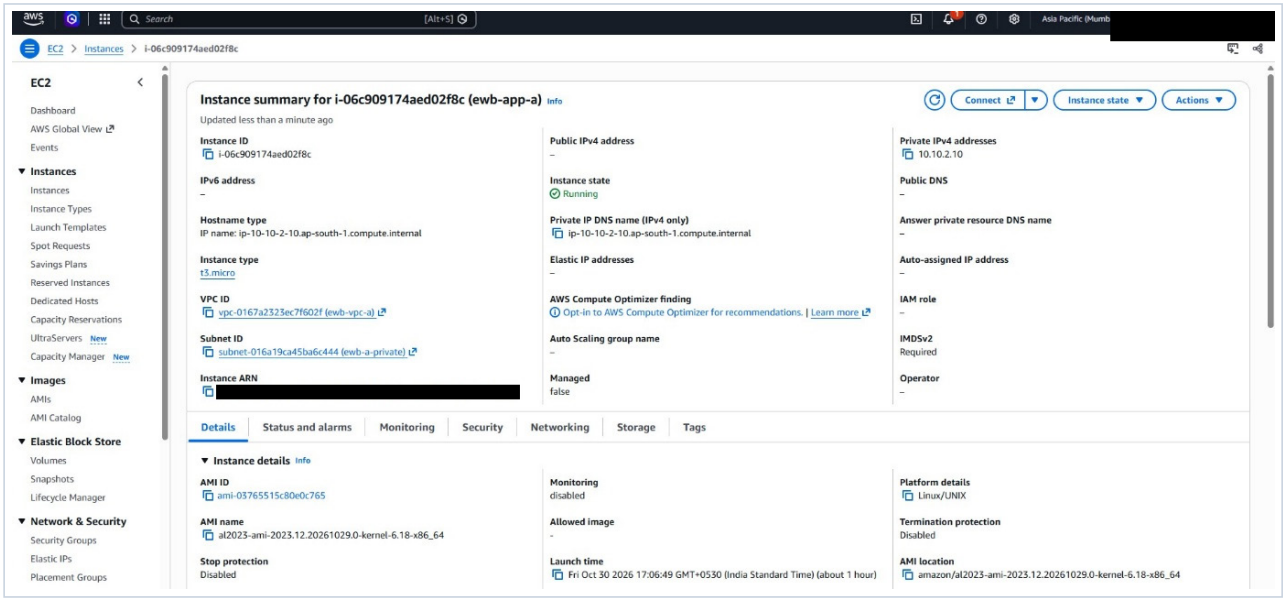


Figure 10. App-A instance details: no public IP

Host / test / result: ewb-app-a / Test 05 Isolation / PASS

What it shows: Public IPv4 address and Public DNS are empty; Private IPv4 is 10.10.2.10 in subnet ewb-a-private of ewb-vpc-a.

Why it is needed: Proves App-A is private: it can only be reached from inside the VPC (through the bastion).

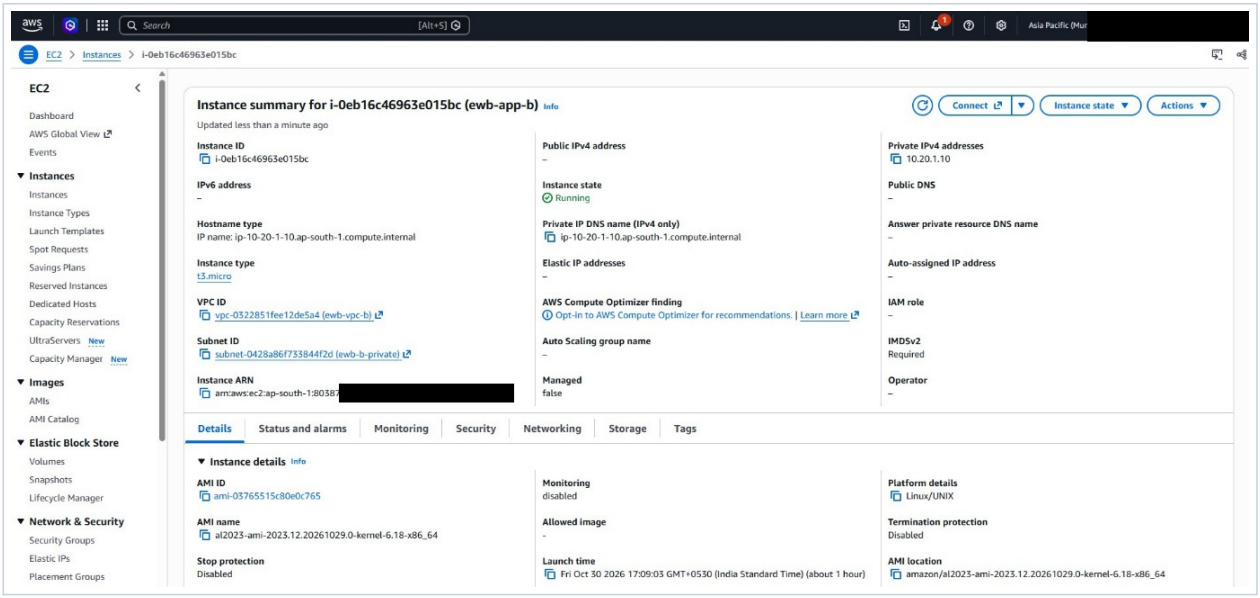


Figure 11. App-B instance details: no public IP

Host / test / result: ewb-app-b / Test 05 Isolation / PASS

What it shows: Public IPv4 address and Public DNS are empty; Private IPv4 is 10.20.1.10 in subnet ewb-b-private of ewb-vpc-b.

Why it is needed: Proves App-B is private. VPC B has no internet gateway at all, so it cannot be reached from the internet.

06 / SECURITY

Security group inbound rules

Only the required inbound traffic is allowed. Outbound stays at the default allow-all; security groups are stateful.

Security group	VPC	Type / port	Source	Purpose
ewb-sg-bastion	A	SSH / TCP 22	my public IPv4 /32 (masked)	Only my laptop can SSH to the bastion
ewb-sg-app-a	A	SSH / TCP 22	10.10.1.10/32 (bastion)	Admin access through the bastion only
ewb-sg-app-a	A	Custom ICMP - Echo request	10.20.1.10/32 (App-B)	Allow ping from App-B
ewb-sg-app-b	B	SSH / TCP 22	10.10.1.10/32 (bastion)	Admin access; App-B sees the bastion private IP
ewb-sg-app-b	B	Custom ICMP - Echo request	10.10.2.10/32 (App-A)	Allow ping from App-A

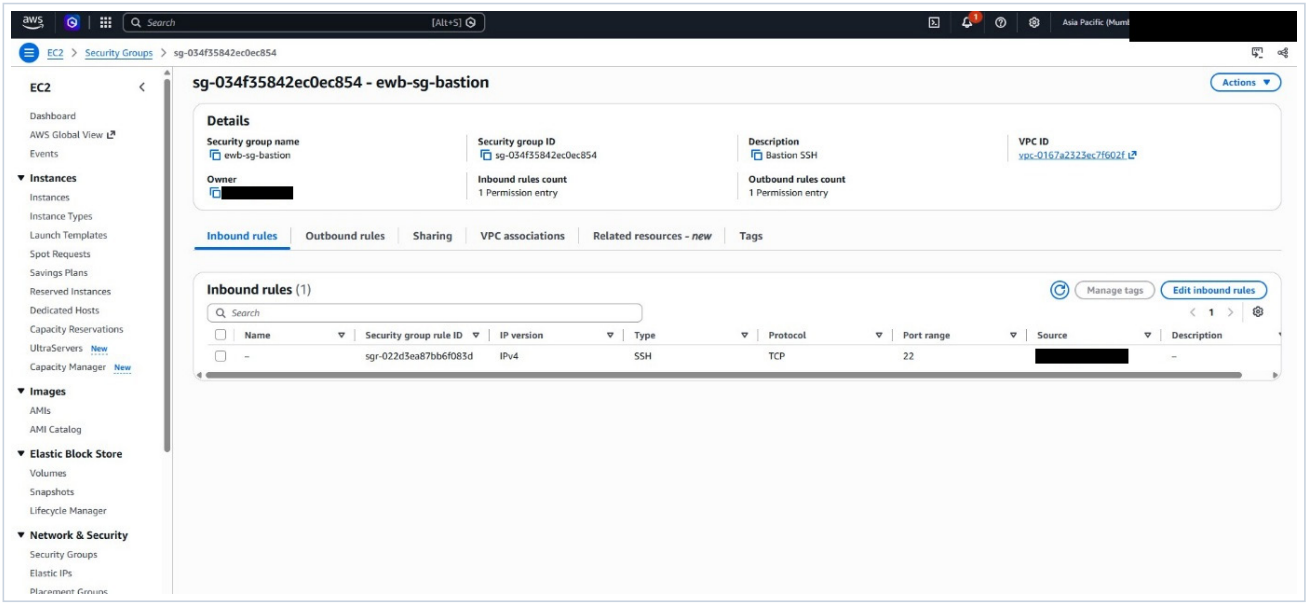


Figure 12. Inbound rules of ewb-sg-bastion

Host / test / result: ewb-sg-bastion / inbound rules / PASS

What it shows: One rule: SSH, TCP port 22, source = my public IP as a /32 (masked in this report). Description "Bastion SSH", VPC ewb-vpc-a.

Why it is needed: The bastion is on the internet, so SSH is limited to one address instead of 0.0.0.0/0. This is the single entry point to the whole lab.

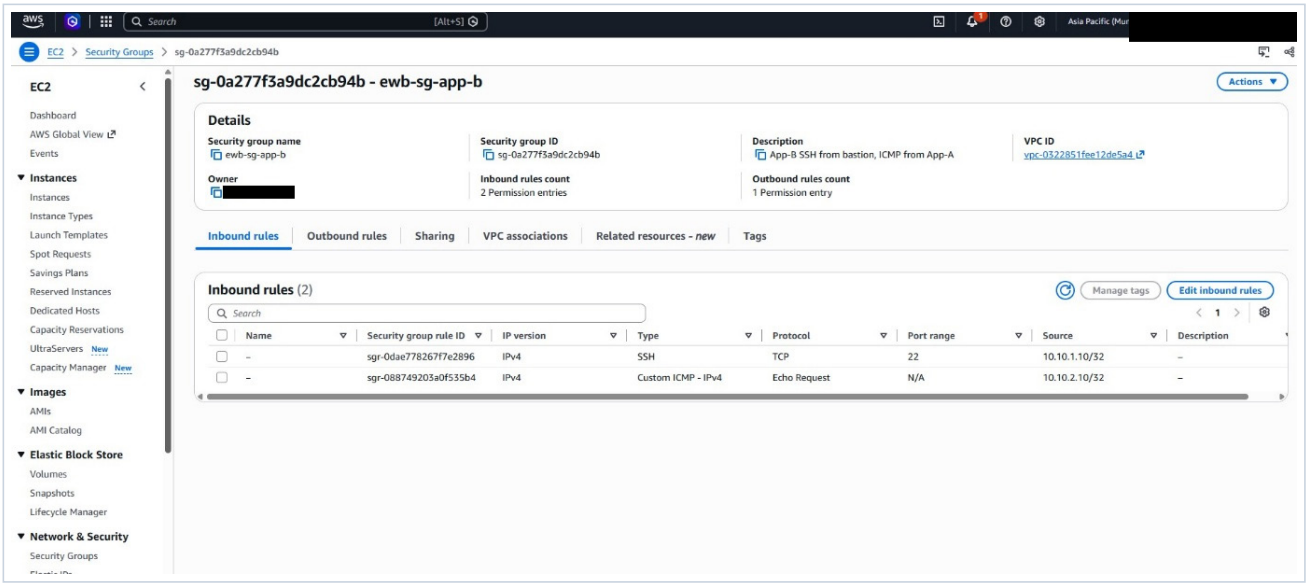


Figure 13. Inbound rules of ewb-sg-app-b

Host / test / result: ewb-sg-app-b / inbound rules / PASS

What it shows: Two rules: SSH TCP 22 from 10.10.1.10/32 (bastion) and Custom ICMP Echo Request from 10.10.2.10/32 (App-A). VPC is ewb-vpc-b.

Why it is needed: The ICMP rule is what allows the ping from App-A; the SSH source is the bastion private IP because ProxyJump makes the bastion open its own SSH connection to App-B.

07 / PEERING

VPC peering connection

Checkpoint 03: the request was created from VPC A and accepted by VPC B.

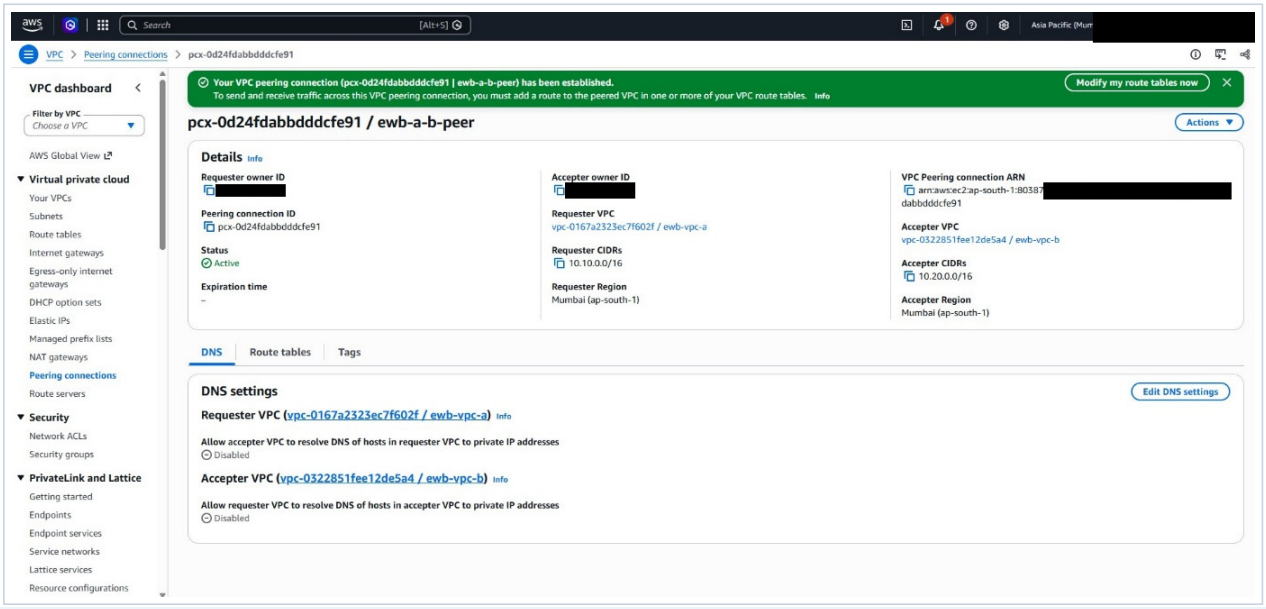


Figure 14. Peering connection ewb-a-b-peer is Active

Host / test / result: VPC > Peering connections / Checkpoint 03 / PASS

What it shows: pcx-0d24fdabbdddcfe91: Status Active; requester VPC ewb-vpc-a (10.10.0.0/16), accepter VPC ewb-vpc-b (10.20.0.0/16), same account and same Region (Mumbai).

Why it is needed: The peering connection is the link between the two VPCs. Active only means the link exists: routes and security groups still decide whether traffic can use it.

Checkpoint 04: routes added on both sides

After peering was active, one route was added to each of the three route tables. The destination is always the other VPC's CIDR and the target is the peering connection.

Route table	Added route	Role
ewb-rt-a-private	`10.20.0.0/16 → pcx-0d24fdabbdddcfe91`	Forward path: App-A to VPC B
ewb-rt-b-private	`10.10.0.0/16 → pcx-0d24fdabbdddcfe91`	Return path: App-B replies to VPC A
ewb-rt-a-public	`10.20.0.0/16 → pcx-0d24fdabbdddcfe91`	Bastion SSH to App-B (ProxyJump)

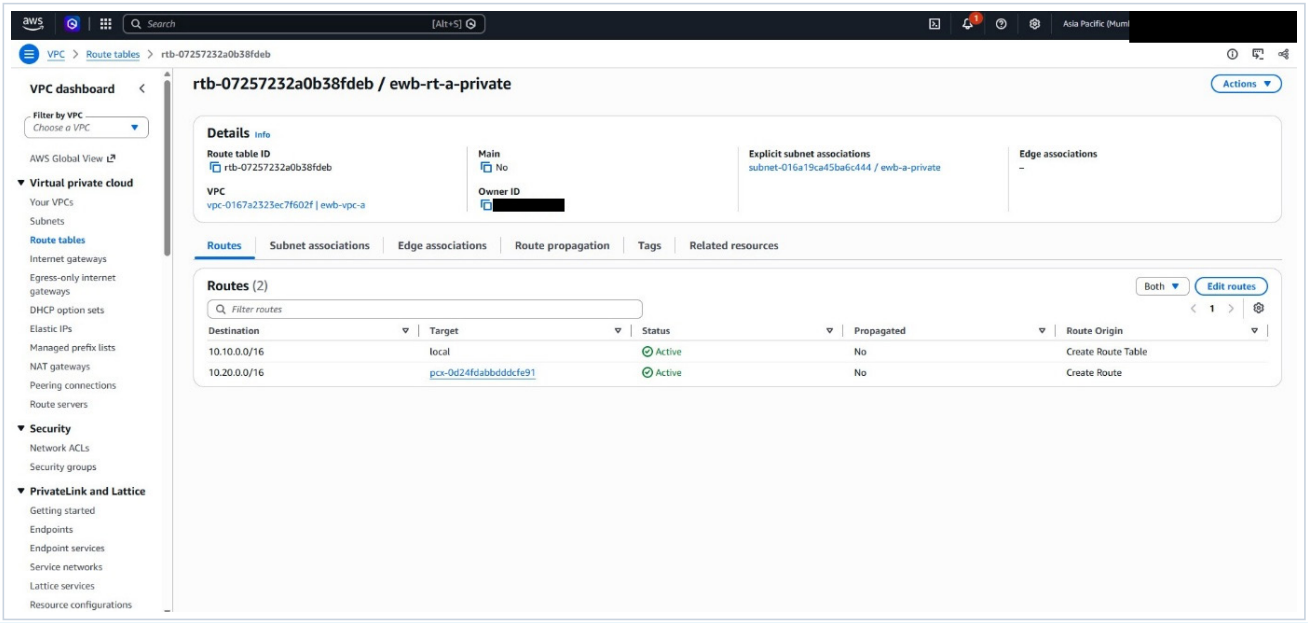


Figure 15. Private route table A after peering

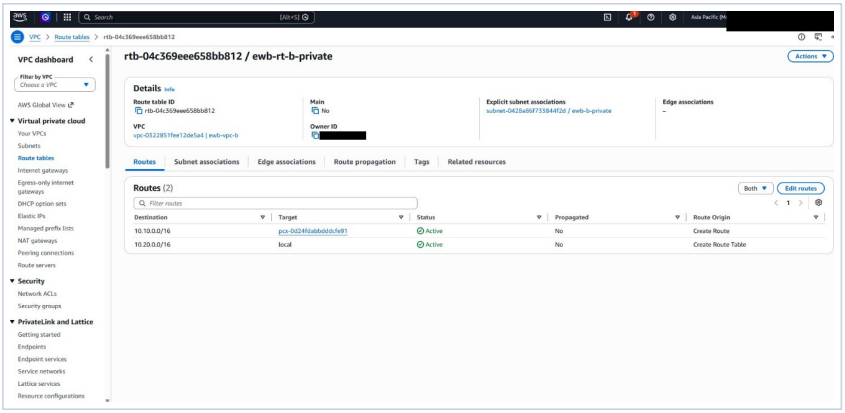


Figure 16. Private route table B after peering

Host / test / result: ewb-rt-b-private / Routes tab / PASS

What it shows: 10.20.0.0/16 → local and 10.10.0.0/16 → pcx-0d24fdabbdddcfe91, both Active.

Why it is needed: Return route: App-B must know how to send replies back to 10.10.2.10, otherwise the ping works one way only and fails.

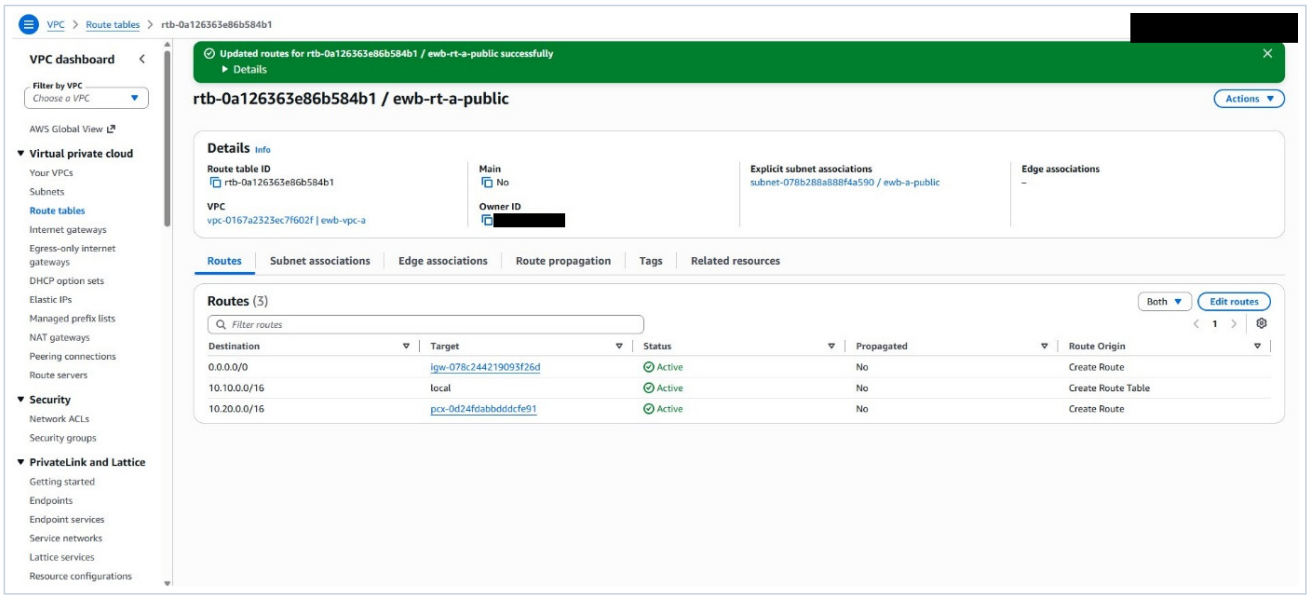


Figure 17. Public route table after peering

Host / test / result: ewb-rt-a-public / Routes tab / PASS

What it shows: 0.0.0.0/0 → igw, 10.10.0.0/16 → local and 10.20.0.0/16 → pcx-0d24fdabbdddcfe91.

Why it is needed: The bastion opens the SSH connection to App-B, so its subnet also needs a route to VPC B. The more specific 10.20.0.0/16 route is chosen instead of the internet default route.

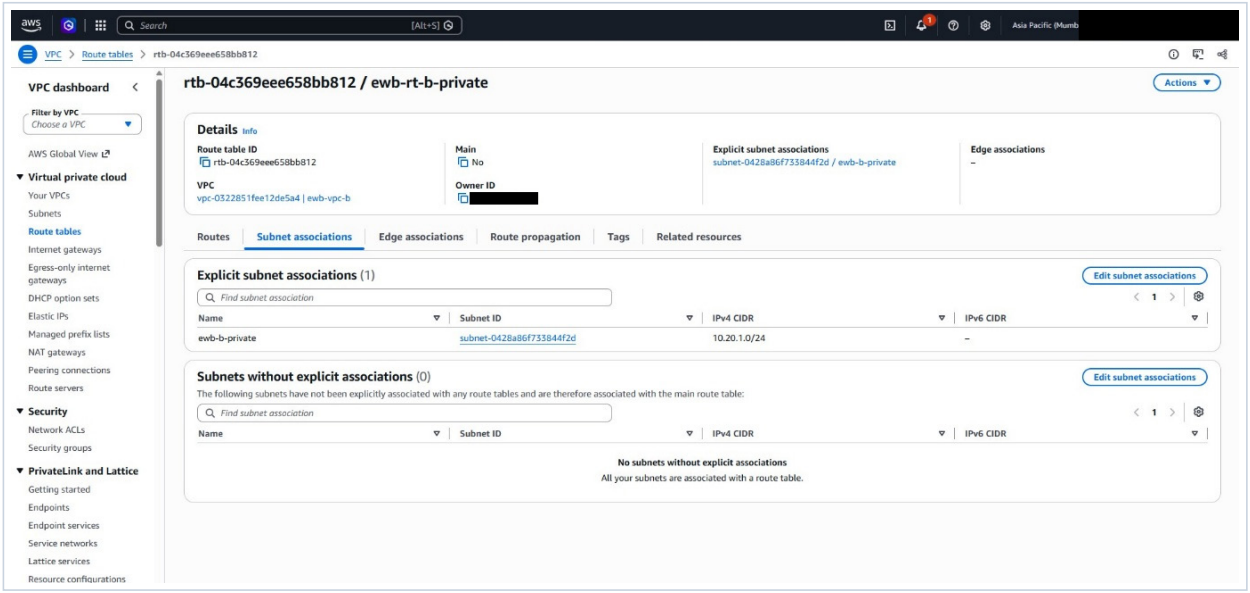


Figure 18. Subnet association of ewb-rt-b-private

Host / test / result: ewb-rt-b-private / Subnet associations tab / PASS

What it shows: Explicit association with ewb-b-private (subnet-0428a86f733844f2d, 10.20.1.0/24).

Why it is needed: Confirms the return route is applied to the subnet where App-B lives.

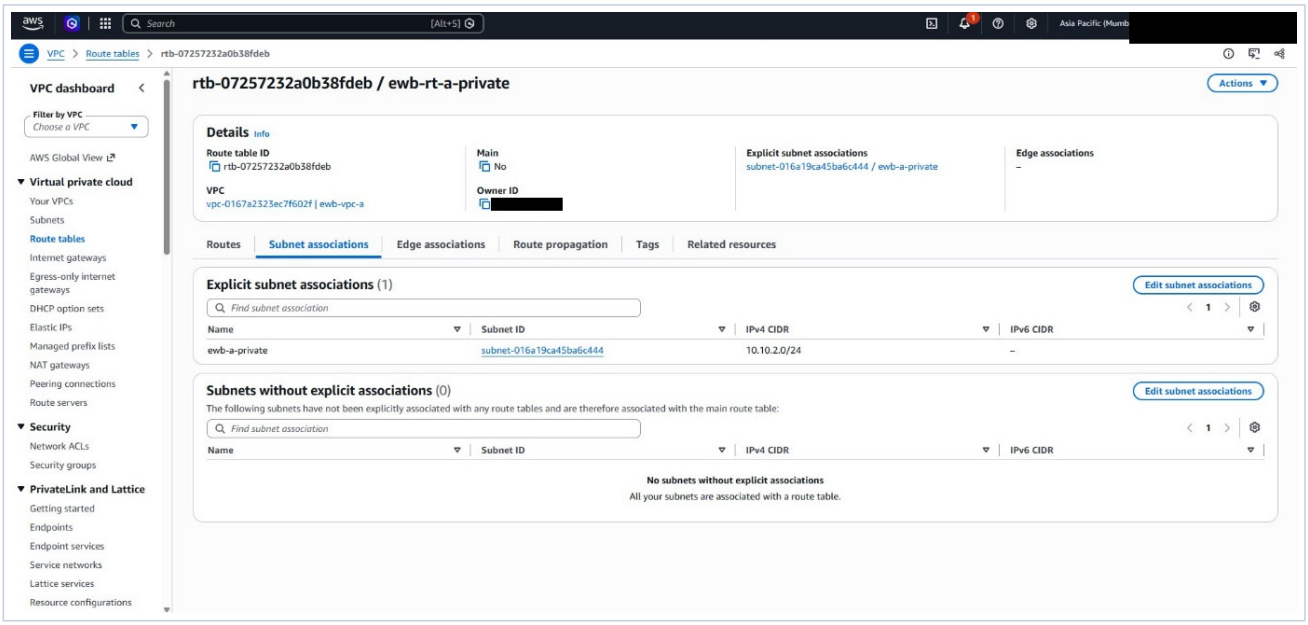


Figure 19. Subnet association of ewb-rt-a-private (after peering)

Host / test / result: ewb-rt-a-private / Subnet associations tab / PASS

What it shows: Still explicitly associated with ewb-a-private (10.10.2.0/24).

Why it is needed: Confirms the forward route is applied to the subnet where App-A lives.

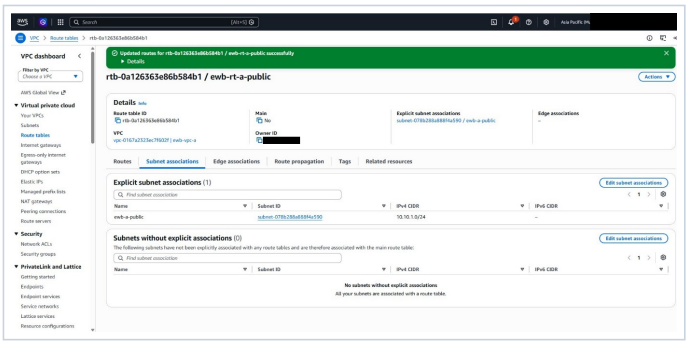


Figure 20. Subnet association of ewb-rt-a-public (after peering)

Host / test / result: ewb-rt-a-public / Subnet associations tab / PASS

What it shows: Still explicitly associated with ewb-a-public (10.10.1.0/24).

Why it is needed: Confirms the bastion subnet uses the table that holds both the IGW route and the peering route.

08 / VERIFICATION

Test results: proving the project works

Every test uses private destination IPs. Output below is from my own terminal.

Test	Where / command	Expected	My result
01 Baseline	App-A, before peering: ping -c 4 10.20.1.10	No replies	4 sent, 0 received, 100% loss: PASS
02 Forward path	App-A, after routes: ping -c 4 10.20.1.10	4 replies, 0% loss	4 received, 0% loss: PASS
03 Reverse path	App-B, after routes: ping -c 4 10.10.2.10	4 replies, 0% loss	4 received, 0% loss: PASS
04 Private SSH	Laptop: ssh ewb-app-b	Shell on App-B via bastion	Connected as ec2-user@ip-10-20-1-10: PASS
05 Isolation	EC2 details of App-A and App-B	No public IPv4	Both empty: PASS
06 Controlled failure	Remove ICMP rule on ewb-sg-app-b, ping, restore, ping	Fail, then recover	100% loss, then 0% loss: PASS

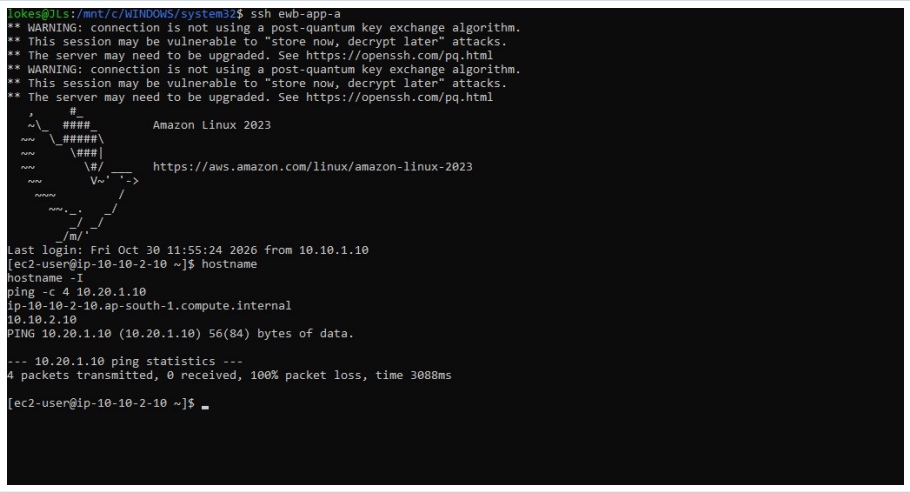


Figure 21. Baseline: ping from App-A to App-B BEFORE peering: FAILED (expected)

Host / test / result: App-A (ip-10-10-2-10) / Test 01 Baseline / 100% packet loss **What it shows:** After ssh ewb-app-a through the bastion, hostname shows ip-10-10-2-10 and hostname -I shows 10.10.2.10. ping -c 4 10.20.1.10 sends 4 packets and receives 0 (100% loss). **Why it is needed:** This negative test proves the two VPCs are isolated by default, so the later success is caused by the peering connection and routes, not by luck.



Figure 22. Forward path: ping from App-A to App-B AFTER peering: SUCCESS

Host / test / result: App-A (10.10.2.10) / Test 02 Forward path / 4 received, 0% loss

What it shows: The command ssh ewb-app-a is visible, then ping -c 4 10.20.1.10 returns 4 replies from 10.20.1.10 (ttl 127, about 0.15 to 0.43 ms) and 0% packet loss.

Why it is needed: Shows that the route 10.20.0.0/16 → pcx on ewb-rt-a-private and the ICMP rule on ewb-sg-app-b work together.

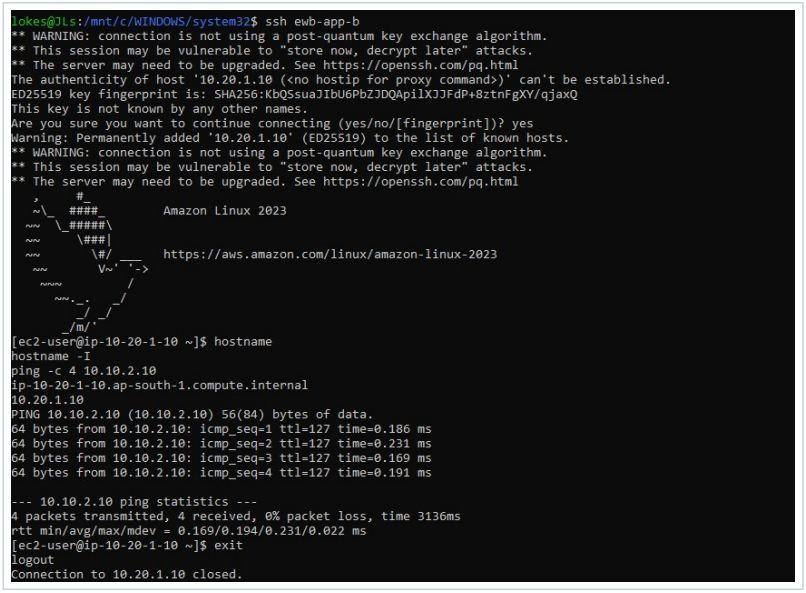


Figure 23. Private SSH to App-B and reverse ping to App-A: SUCCESS

Host / test / result: Laptop to App-B (10.20.1.10) / Test 04 and Test 03 / shell opened, 4 received, 0% loss

What it shows: ssh ewb-app-b from the laptop reaches App-B through the bastion (ProxyJump). Inside App-B, hostname is ip-10-20-1-10 and ping -c 4 10.10.2.10 returns 4 replies with 0% packet loss.

Why it is needed: The SSH shell is the private SSH proof (the SSH source rule is the bastion's private IP). The ping from the other side proves the return route works.

09 / CONTROLLED FAILURE

Controlled failure and recovery

Remove only the ICMP echo rule from ewb-sg-app-b, observe the failure, restore the exact rule and observe recovery. The SSH rule stays in place.

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3138ms

[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=127 time=0.362 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=127 time=0.331 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=127 time=0.314 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=127 time=0.337 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3127ms
rtt min/avg/max/mdev = 0.314/0.336/0.362/0.017 ms
```

Figure 24. Ping fails with the ICMP rule removed, then succeeds after restoring it Host / test / result: App-A (10.10.2.10) to App-B (10.20.1.10) / Test 06 / first run 100% loss, second run 0% loss

What it shows: First ping -c 4 10.20.1.10: 4 transmitted, 0 received, 100% packet loss. After the rule was restored, the same command returns 4 replies and 0% packet loss.

Why it is needed: Shows that the security group (not just the route) decides whether ping can reach App-B, and that restoring the same rule recovers connectivity.

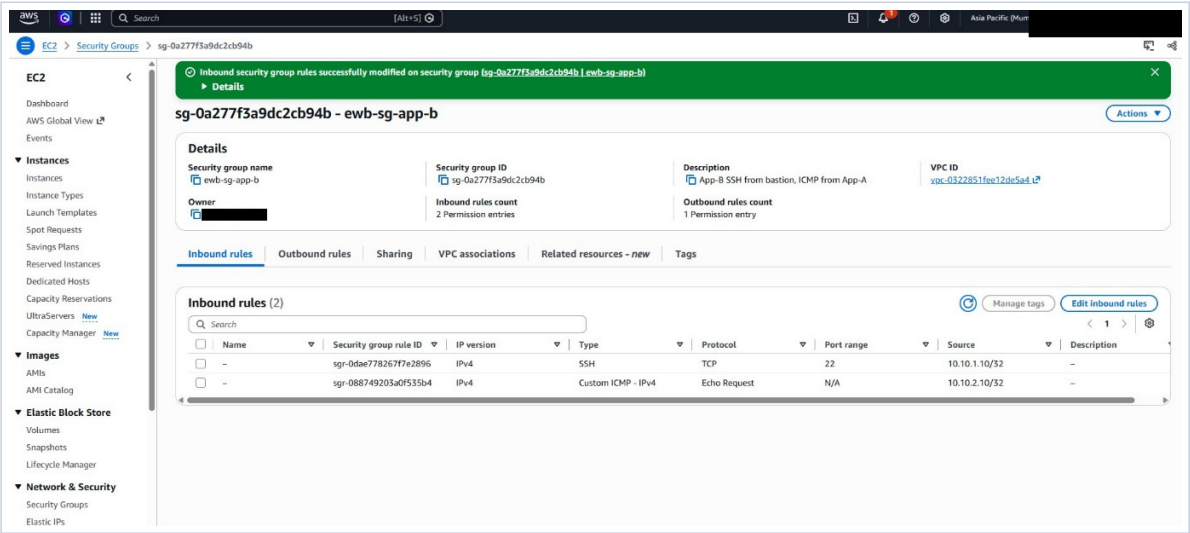


Figure 25. Security group ewb-sg-app-b after the rule change

Host / test / result: ewb-sg-app-b / inbound rules / confirmation banner

What it shows: Green banner “Inbound security group rules successfully modified”; the group again lists SSH 22 from 10.10.1.10/32 and Custom ICMP Echo Request from 10.10.2.10/32.

Why it is needed: Documents that the inbound rules were edited and that the exact original ICMP rule is back in place.

Explanation. While the ICMP rule was removed, packets from App-A still found the route and the peering connection, but the security group on App-B dropped them: there was no allow rule for the echo request, so no reply was sent. When the same rule (Custom ICMP – IPv4, Echo request, source 10.10.2.10/32) was added back, the echo request was allowed again and replies resumed. Routing, peering and security rules are three separate layers, and all must work.

10 / EXPLANATION

Forward path, return path and viva answers

Forward path (App-A → App-B). App-A (10.10.2.10) sends an echo request to 10.20.1.10. The route table ewb-rt-a-private has 10.20.0.0/16 → pcx, so the packet goes over the peering connection into VPC B. The security group ewb-sg-app-b allows ICMP echo request from 10.10.2.10/32, so App-B receives it.

Return path (App-B → App-A). App-B sends the reply to 10.10.2.10. The route table ewb-rt-b-private has 10.10.0.0/16 → pcx, so the reply goes back over the peering connection. Security groups are stateful, so the reply is allowed automatically.

Questions I should be ready to explain

1. Why is the return route needed?
Routing is not automatic in both directions. Without 10.10.0.0/16 → pcx in VPC B's route table, App-B would receive the request but have no path to send the reply back, so the ping would fail.

2. Why is the App-B SSH source the bastion private IP?
With ProxyJump the bastion opens a new SSH connection to App-B. App-B therefore sees the source 10.10.1.10, not my laptop's public IP, so the rule must allow 10.10.1.10/32.

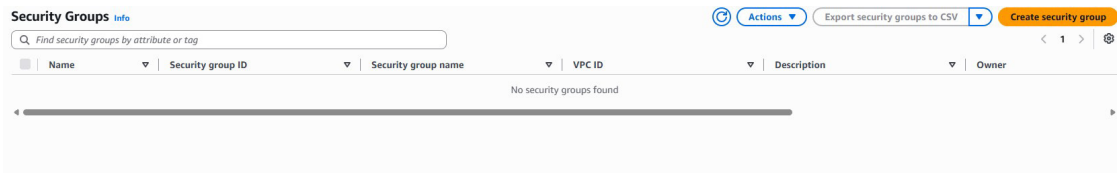
3. Why does Active peering not guarantee a successful ping?
Active only means the connection is established. Packets still need routes in both VPCs (forward and return) and security group rules that allow the traffic.

4. Can VPC B use VPC A's internet gateway over peering?
No. Peering is one-to-one and non-transitive; an internet gateway, NAT or edge connection in one VPC cannot be used by the peered VPC. VPC B has no internet access.

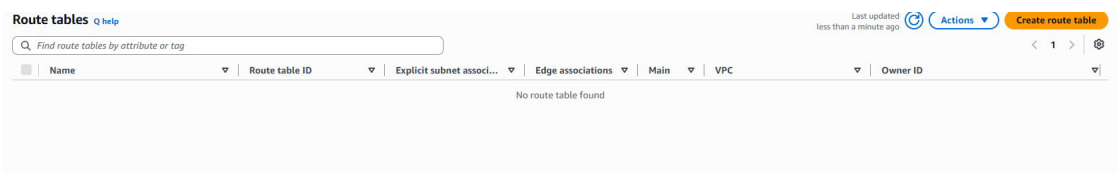
11 / CLEANUP

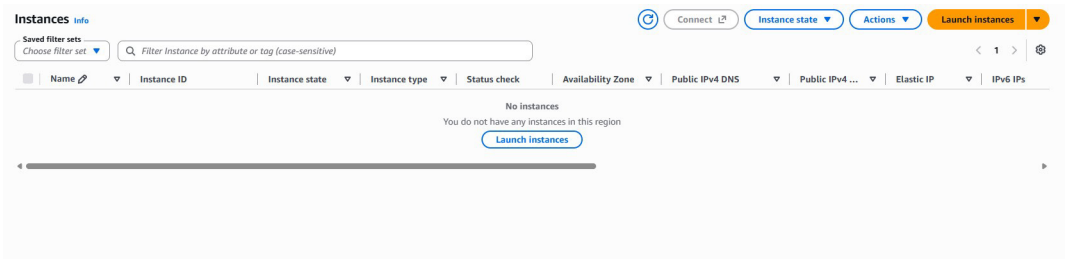
Cleanup evidence

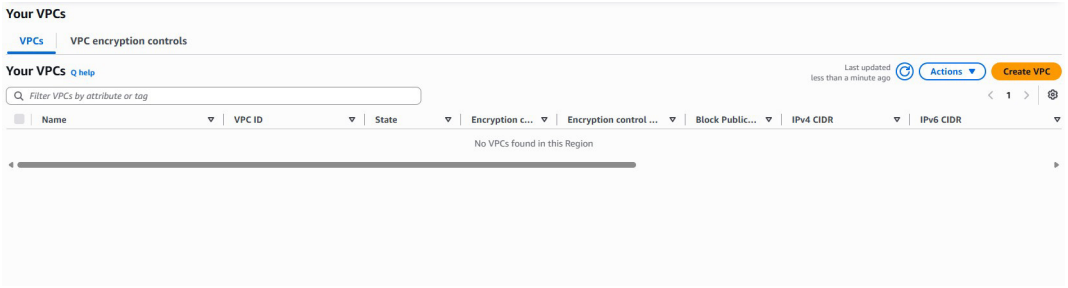
Resources created for this project are removed in this order: terminate the three instances, confirm their volumes are deleted, delete the peering routes and ewb-a-b-peer, delete the security groups, subnets and custom route tables, detach and delete ewb-igw-a, delete both VPCs, delete the ewb-lab key pair and the local .pem file.

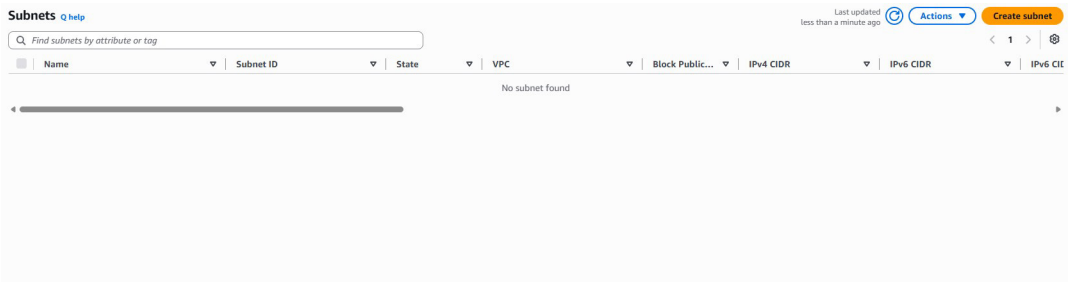












12 / REFLECTION

What I learned

- A VPC peering connection only creates the link; both route tables need a route to the other VPC's CIDR, and the security groups must allow the traffic.
- A baseline failure test is important: it proves that the success after peering is caused by my configuration.
- Private servers can be managed safely through one bastion host using SSH ProxyJump, without copying the private key to any instance.
- Peering needs non-overlapping CIDRs and is not transitive, so VPC B cannot use VPC A's internet gateway.
- Security groups are stateful and can be tightened to /32 sources; removing one rule broke the ping and restoring it fixed it.

No keys, passwords or tokens are included. Account number and public IPs are masked.